Assignment 3 Bridge Pattern

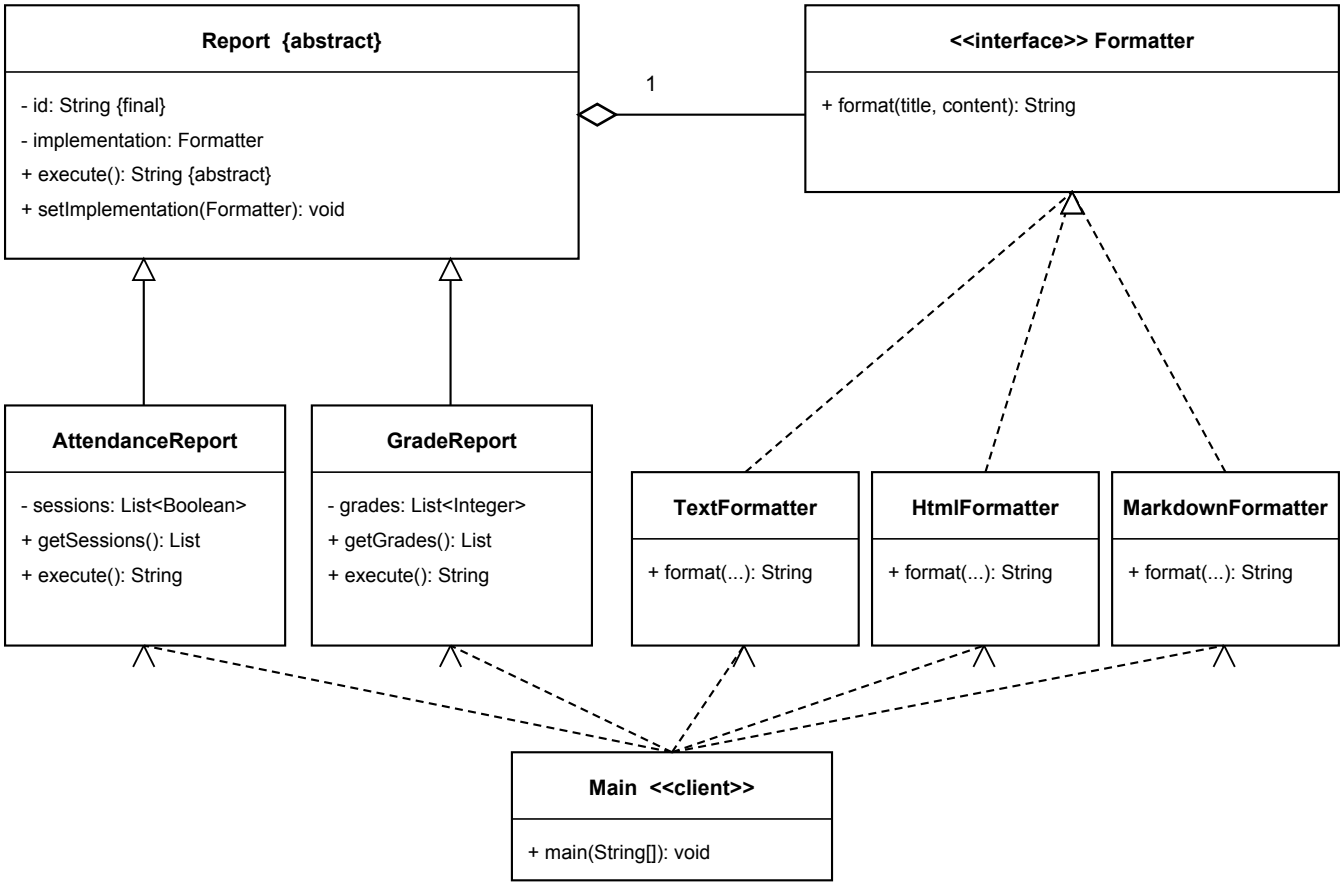
Topic C Reports | Prepared for Nurdaulet Mukhitden | Group SE-2537

AI-generated study reference. This material is for learning and does not establish independent student authorship.

Two independently changing dimensions

AttendanceReport calculates the attended fraction; GradeReport calculates an average. Formatter decides how the calculated content is presented. A report stores an interface-typed Formatter reference supplied through its constructor. New report types and new output formats can therefore be added independently.

The fixed attendance data contains three attended sessions out of four, so execute() produces 75%. Grades 70, 80 and 90 produce an average of 80. Text, HTML and Markdown output visibly differ while preserving those calculations.



Hollow triangles: inheritance/realization. Dashed arrows: dependency. Hollow diamond: aggregation.

Runtime replacement

T5 keeps the original AttendanceReport reference, executes it with TextFormatter, then calls setImplementation(new HtmlFormatter()) on that same object. Reference equality uses ==. The check also compares the ID, immutable session data and both actual outputs. Matching an ID alone would not prove object identity.

Five Clean Code examples

Separation of responsibilities

src/AttendanceReport.java:18

```
        return format("Attendance", attended + " of " + sessions.size()
                + " attended (" + percent + "%)");
```

The report calculates domain content and delegates presentation through the base class. It does not build HTML or select a concrete formatter.

Meaningful names

src/Report.java:14

```
    public final void setImplementation(Formatter implementation) {
        this.implementation = Objects.requireNonNull(implementation);
    }
```

The method name describes exactly what changes. The interface parameter expresses the supported capability without coupling the report to a concrete class.

Small focused methods

src/TextFormatter.java:3

```
    public String format(String title, String content) {
        return "TEXT " + title + " | " + content;
    }
```

This operation only creates the plain-text representation. Calculation and demonstration checks live in other classes.

Avoiding duplicated delegation

src/Report.java:17

```
    protected final String format(String title, String content) {
        return implementation.format(title, content);
    }
```

Both report subclasses use this one delegation method. The reference-handling workflow is not copied into every subclass.

Encapsulation of state

src/AttendanceReport.java:9

```
        this.sessions = List.copyOf(sessions);
```

List.copyOf prevents callers from changing stored domain data through a mutable input collection. The ID is also final, so switching a formatter does not replace report identity or input data.

Verification and extension

Check	Observed result
T1	AttendanceReport + TextFormatter; 75%
T2	AttendanceReport + HtmlFormatter; 75%
T3	GradeReport + TextFormatter; 80
T4	GradeReport + HtmlFormatter; 80
T5	Same object and unchanged state; TEXT changes to HTML
T6	AttendanceReport + MarkdownFormatter; 75%
T7	GradeReport + MarkdownFormatter; 80

The submitted local program was compiled with Amazon Corretto 17.0.13 using --release 17. All seven checks calculate PASS or FAIL from actual results and finish with SUMMARY: 7/7 PASS. demo-output.txt is captured execution output.

Independent extension

The two-by-two version and T5 were committed first. The extension adds MarkdownFormatter.java and changes Main.java to run T6 and T7. The source diff contains only those two paths. Existing reports, Formatter, TextFormatter and HtmlFormatter remain byte-for-byte unchanged.

Base commit 7552e89c78c9e38017ad32ba60dd45b04794d864

Verified source commit 7c464d43c7518d693ecf8270e44bb8f06e7d64f0

Design trade-off and comparison

Bridge adds indirection and extra classes. That cost is justified here by independently changing calculations and presentation. Adapter addresses a compatibility mismatch with an existing interface; this application is deliberately designed around two dimensions and an agreed Formatter contract.

Before any course submission

Repository: https://github.com/unknow9error/sdp-3. The original deadline was 4 October 2026 at 23:59; the instructions state that late submissions receive zero without an approved exception. A mandatory defense remains. Study and independently implement the design under the course AI rules before representing work as your own. No grade is claimed.

References

Course Assignment_3_Bridge_Pattern_Instructions.pdf, sections 1-9. Refactoring.Guru, Bridge: https://refactoring.guru/design-patterns/bridge (accessed 17 October 2026).